

// Jivops

Guía completa — 30 lecciones

Prometheus · Grafana · Alerting · Logging · Tracing y SLOs · Madurez operativa

Contenido

Módulo 1: Prometheus avanzado

- Lección 01 — Arquitectura de Prometheus en producción
- Lección 02 — PromQL avanzado — funciones de agregación
- Lección 03 — Recording rules
- Lección 04 — Federación de Prometheus
- Lección 05 — Exporters personalizados

Módulo 2: Grafana avanzado

- Lección 06 — Grafana: variables de dashboard
- Lección 07 — Grafana: paneles avanzados (heatmaps, histogramas)
- Lección 08 — Grafana: alerting nativo
- Lección 09 — Dashboards como código (Grafonnet/JSON)
- Lección 10 — Grafana: permisos y organizaciones

Módulo 3: Alerting e incidentes

- Lección 11 — Alertmanager: routing avanzado
- Lección 12 — Alertmanager: inhibición y silencios
- Lección 13 — Runbooks y respuesta a incidentes
- Lección 14 — On-call y escalado de alertas
- Lección 15 — Postmortems sin culpa (blameless)

Módulo 4: Logging avanzado

- Lección 16 — Logging: Loki arquitectura
- Lección 17 — LogQL — consultas avanzadas
- Lección 18 — Retención y costos de logs
- Lección 19 — Correlación logs-métricas-trazas
- Lección 20 — Alertas basadas en logs

Módulo 5: Tracing y SLOs

- Lección 21 — Tracing distribuido con Tempo
- Lección 22 — Instrumentación con OpenTelemetry
- Lección 23 — SLI: definir indicadores de nivel de servicio
- Lección 24 — SLO: definir objetivos y error budgets
- Lección 25 — Dashboards de SLO

Módulo 6: Madurez operativa

Lección 26 — Capacity planning con métricas históricas

Lección 27 — Detección de anomalías

Lección 28 — Costos de observabilidad (cardinalidad)

Lección 29 — Auditoría y cumplimiento en monitoreo

Lección 30 — Proyecto final: stack de observabilidad completo en producción

Módulo 1: Prometheus avanzado

Lección 01 — Arquitectura de Prometheus en producción

Prometheus obtiene métricas por pull (scraping) periódico a endpoints /metrics. Este modelo facilita detectar si un servicio dejó de responder, ya que el propio Prometheus sabe de inmediato si el scrape falla.

```
scrape_interval: 15s
```

Lección 02 — PromQL avanzado — funciones de agregación

rate() calcula la tasa de incremento por segundo de un counter en una ventana de tiempo. Es más útil que el valor crudo, porque un counter solo sube y se resetea al reiniciar el proceso.

```
rate(http_requests_total[5m])
```

Lección 03 — Recording rules

Las recording rules precálculan consultas costosas y las guardan como nuevas métricas, mejorando el rendimiento de dashboards que reutilizan esa misma consulta repetidamente.

Lección 04 — Federación de Prometheus

La federación agrega métricas de varios servidores Prometheus locales en uno central, útil cuando se tienen varios clusters o regiones y se quiere una vista centralizada.

```
/federate?match[]={job="api"}
```

Lección 05 — Exporters personalizados

Un exporter expone métricas de un sistema en formato Prometheus. Se escribe uno propio cuando el sistema a monitorear no tiene ya un exporter de la comunidad.

Módulo 2: Grafana avanzado

Lección 06 — Grafana: variables de dashboard

Las variables permiten filtrar un dashboard dinámicamente (por servicio, entorno, etc.), evitando duplicar el mismo dashboard para cada caso.

```
$variable
```

Lección 07 — Grafana: paneles avanzados (heatmaps, histogramas)

Un heatmap visualiza la distribución de valores (como latencias) a lo largo del tiempo, revelando outliers que un simple promedio oculta.

Lección 08 — Grafana: alerting nativo

El alerting nativo de Grafana define alertas directamente sobre paneles y las enruta a contact points. Para routing e inhibición más complejos, Alertmanager sigue siendo la opción más potente.

Lección 09 — Dashboards como código (Grafonnet/JSON)

Versionar dashboards como JSON o Grafonnet permite revisar cambios, reutilizar plantillas y desplegarlos vía CI/CD, evitando perder el historial de quién modificó qué.

Lección 10 — Grafana: permisos y organizaciones

Las organizaciones aíslan dashboards, datasources y usuarios entre equipos o clientes distintos, útil si una plataforma sirve dashboards a múltiples clientes.

Módulo 3: Alerting e incidentes

Lección 11 — Alertmanager: routing avanzado

El routing decide a qué receptor (Slack, email, PagerDuty) se envía cada alerta según sus labels, permitiendo priorizar lo crítico sin saturar de ruido.

Lección 12 — Alertmanager: inhibición y silencios

La inhibición suprime automáticamente alertas relacionadas con una causa raíz ya identificada; el silencio es una supresión manual y temporal definida por una persona.

Lección 13 — Runbooks y respuesta a incidentes

Un runbook es una guía paso a paso para diagnosticar y mitigar un incidente conocido. Enlazarlo directamente desde la alerta acelera la respuesta al evitar buscar el procedimiento a mitad del incidente.

Lección 14 — On-call y escalado de alertas

Una política de escalado asegura que si la persona de guardia no responde en X minutos, la alerta pase a la siguiente, evitando un punto único de fallo humano.

Lección 15 — Postmortems sin culpa (blameless)

Un postmortem blameless busca entender la causa raíz para prevenir el incidente, sin señalar culpables individuales — esto fomenta que la gente reporte problemas antes en vez de ocultarlos.

Módulo 4: Logging avanzado

Lección 16 — Logging: Loki arquitectura

Loki indexa solo metadatos (labels), no el contenido completo del log, lo que lo hace más barato y eficiente en recursos que indexar el texto completo.

Lección 17 — LogQL — consultas avanzadas

Combinar un filtro por label con un filtro de texto reduce primero el volumen de datos, y luego filtra el texto sobre ese subconjunto más pequeño y rápido.

```
{app="api"} |= "error"
```

Lección 18 — Retención y costos de logs

No conviene guardar todos los logs para siempre: el costo crece indefinidamente. Una política por niveles (detalle a corto plazo, resúmenes a largo plazo) balancea costo y necesidad de auditoría.

Lección 19 — Correlación logs-métricas-trazas

Un trace_id o request_id compartido entre logs, métricas y trazas permite correlacionar el mismo evento en los tres pilares de observabilidad.

Lección 20 — Alertas basadas en logs

Alertar sobre logs tiene sentido cuando el evento (ej. un patrón de error específico) no está expuesto como métrica numérica.

Módulo 5: Tracing y SLOs

Lección 21 — Tracing distribuido con Tempo

Un span es una unidad de trabajo individual dentro del recorrido de una petición. El conjunto de spans de una petición forma un trace completo, mostrando cuánto tiempo tomó cada servicio.

Lección 22 — Instrumentación con OpenTelemetry

OpenTelemetry es un estándar abierto de instrumentación, independiente del backend de observabilidad usado después, reduciendo el vendor lock-in.

Lección 23 — SLI: definir indicadores de nivel de servicio

Un SLI es una métrica cuantitativa que refleja la experiencia real del usuario (ej. porcentaje de peticiones con latencia bajo 200ms).

Lección 24 — SLO: definir objetivos y error budgets

El error budget es el margen de fallos permitido antes de incumplir el SLO. Si el budget está agotado (ej. SLO 99.9%, budget 0.1%), se prioriza estabilidad sobre nuevas features.

Lección 25 — Dashboards de SLO

Un dashboard de SLO muestra el SLI actual, el objetivo y el error budget restante, dando a todo el equipo — no solo SRE — visibilidad para decidir con confianza si se puede seguir desplegando.

Módulo 6: Madurez operativa

Lección 26 — Capacity planning con métricas históricas

El capacity planning usa tendencias históricas de uso de recursos para anticipar cuándo se necesitará más infraestructura, evitando reaccionar recién cuando el sistema colapsa.

Lección 27 — Detección de anomalías

A diferencia de un umbral fijo, la detección de anomalías compara el comportamiento actual contra un patrón histórico, detectando por ejemplo tráfico inusualmente bajo en horas pico.

Lección 28 — Costos de observabilidad (cardinalidad)

Usar labels de alta cardinalidad (como `user_id`) puede generar millones de series de tiempo distintas, causando explosión de memoria y degradación del rendimiento.

Lección 29 — Auditoría y cumplimiento en monitoreo

El control de acceso a dashboards sensibles ayuda a cumplir requisitos de auditoría, dando trazabilidad de quién accedió a qué datos.

Lección 30 — Proyecto final: stack de observabilidad completo en producción

Se integra todo lo anterior: Prometheus, Grafana, Loki y Tempo, con Alertmanager bien configurado, SLOs definidos, runbooks enlazados y una política de on-call clara — el nivel esperado de un stack maduro.